

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Chapter 9, Sections 9.1 to 9.4 (included)
- **Estimated reading time:** 1 hour

Module 4: Lesson 1

Recurrent Neural Networks (RNNs)



Outline

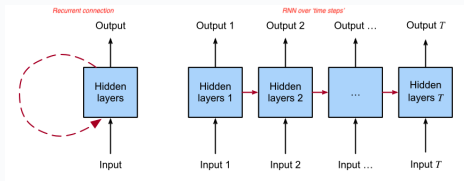
- ▶ What are RNNs and why are they useful?
- ▶ Working with sequences
- ▶ Latent auto-regressive models
- ▶ Recurrent Neural Networks

What are RNNs and why are they useful?

We already know how to build neural networks that work with data of different lengths and characteristics such as images, but we haven't dealt with sequential data yet:

- What should we do when the order of the input data matters for the prediction?
 - Sequence of images → Video!
 - Sequence of words → Text! → Natural Language Processing (NLP)

Recurrent Neural Networks (RNNs) are deep learning models that capture the dynamics of sequences via recurrent connections—i.e., cycles in the network.



Working with sequences

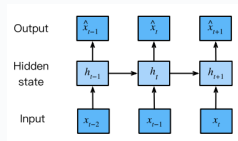
The fact that a specific sequence of input data matters for the prediction of the next is very intuitive:

- what is behind the auto-complete feature of your phones when you text?

However, working with sequences in a deep neural network has some challenges. In order to properly understand what RNNs do, let's start from a baseline: **autoregressive (AR) models**.

- ▶ AR models aim to predict a future value x_t given the past values of this variable: $\mathbb{E}[x_t | x_{t-1}, \dots, x_{t-n}]$
- ▶ Problem #1: How many past observations do we need? → in Latent AR models h_t summarizes the past

$$\hat{x}_t = \mathbb{P}(x_t | h_t), \text{ where } h_t = g(h_{t-1}, x_{t-1})$$

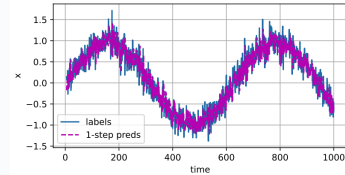
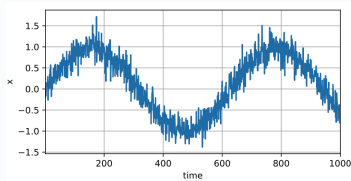


- ▶ Problem #2: Markov condition? → Given recent history, future conditionally independent of the past?

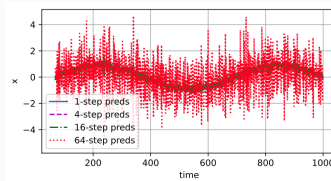
$$P(x_1, \dots, x_T) = P(x_1) \prod_{t=2}^T P(x_t | x_{t-1}, \dots, x_1)$$

Working with sequences

Suppose we want to predict the next logical step of a sin function with some noise like the one on the left. Then, we can use linear regression to predict the one-step-ahead values:



How about predicting 4 steps ahead? and 16? 64?



RNNs for NLP

One of the main uses of RNNs nowadays is in natural language processing (NLP). Unfortunately, the scope for applying NLP into quantitative finance problems is, for now, very limited.

We will not see in depth the use of RNNs for NLP during this course, but please note that the functioning of the whole network model that we will learn will be equivalent to that for NLP.

There is just one thing within the NLP framework that will need to be taken care of: **tokenization**:

- ▶ Tokens are the atomic (indivisible) units of text.
- ▶ A token can be entire words, characters, or even sentences.
- ▶ Still, tokens are simply strings that need to be integrated within a **vocabulary** in numerical form
- ▶ Once we have transformed text into sequence data we can simply train language models as estimates of the joint probability of a text sequence!

You can learn more about this in sections 9.2 and 9.3 in chapter 9 of Zhang et al.'s book *[Dive into Deep Learning](#)*.

Recurrent Neural Networks (RNNs)

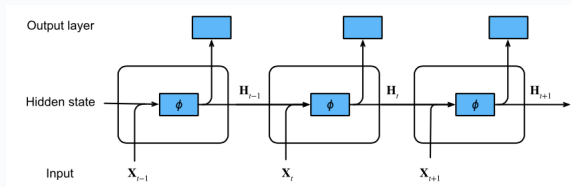
RNNs are neural networks **with hidden states**, similar to the latent AR model.

- First, let's revisit briefly how an NN **without hidden states** (e.g., MLP) worked:

$$H = \phi(XW_{xh} + b_h) \rightarrow O = HW_{hq} + b_q$$

- In an NN **with hidden states**, we save the hidden layer output (H_{t-1}) with a weight (W_{hh}) to account for the sequence. So, for time step t :

$$H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h) \rightarrow O_t = H_t W_{hq} + b_q$$



Summary of Lesson 1

In Lesson 1, we have:

- ▶ Introduced RNN
- ▶ Discussed models with hidden states
- ▶ Basics of building a RNN

⇒ **References for this lesson:**

Chapter 9, Sections 9.1 to 9.4 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement all these novel things in practice by building our first recurrent neural network.

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Chapter 9: Sections 9.5 and 9.6
- **Estimated reading time:** 20 minutes

Module 4: Lesson 2

RNN implementation



Outline

- ▶ Implementing an RNN
- ▶ Predicting stock prices with RNNs

Implementing a RNN in practice

In lesson 2, we will implement an RNN in practice for the first time. As you will see in the accompanying notebook, most of the process (at least from a coding perspective) is identical to what we have done before for other neural network architectures:

1. Gather data
2. Define samples: validation, train, and test
→ Note here that we will also need to define the 'window' for the sequence of inputs we want to consider!
3. Defining the model architecture

```
model = tf.keras.models.Sequential([
    tf.keras.layers.SimpleRNN(50, return_sequences=True, input_shape=(X_train.shape[1], n_features)),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=True),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=True),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=False),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10),
    tf.keras.layers.Dense(1)
])
```

4. Training the model!

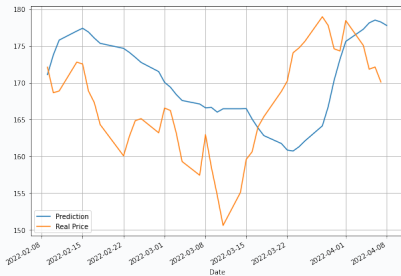
Predicting stock prices with RNNs

We will practice our implementation of RNNs with a prediction problem already familiar to us: **future evolution of a stock!**

- ▶ In this case, we will predict **prices** (rather than returns). Indeed, it makes sense that prices follow a sort of 'sequence' so that we can predict the next day's price by learning the sequence with RNNs.
- ▶ We build an RNN model to predict $d + 1$ price given a sequence of up to 20 past obs. on input prices.

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 20, 50)	2600
dropout (Dropout)	(None, 20, 50)	0
simple_rnn_1 (SimpleRNN)	(None, 20, 50)	5050
dropout_1 (Dropout)	(None, 20, 50)	0
simple_rnn_2 (SimpleRNN)	(None, 20, 50)	5050
dropout_2 (Dropout)	(None, 20, 50)	0
simple_rnn_3 (SimpleRNN)	(None, 50)	5050
dropout_3 (Dropout)	(None, 50)	0
dense (Dense)	(None, 10)	510
dense_1 (Dense)	(None, 1)	11

Total params: 18,271
Trainable params: 18,271
Non-trainable params: 0



Summary of Lesson 2

In Lesson 2, we:

- ▶ Implemented our first RNN
- ▶ Tested the performance of RNNs in predicting stock prices

⇒ **References for this lesson:**

Chapter 9, Sections 9.5 and 9.6 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: Go to the accompanying Python Notebook to implement your first RNN!

In the next lesson, we will improve different aspects of the network and also explore in depth the details of forward and backpropagation in RNNs.

DEEP LEARNING FOR FINANCE

MODULE 4 | LESSON 2

Implementing univariate Recurrent Neural Networks (RNN)

Reading Time	40 minutes
Prior Knowledge	RNN, Market timing, Deep Neural Networks, Dropout
Keywords	RNN, Market timing

1. Introduction and data gathering

In our first practical contact with Recurrent Neural Networks (RNNs) we will implement a model to predict stock **prices** (rather than returns, as we have seen in previous modules). This has an importance, because prices are closer to be interpreted as a sequences than returns. This makes them potentially more predictive using RNNs than returns.

As usual, we will start by downloading price data from Yahoo! finance. Note that we take prices from just one stock, our usual suspect Apple Inc., to explore the time-series univariate prediction case. In lesson 4 we will extend this to a multivariate case.

```
In [1]: import numpy as np
import pandas as pd
import yfinance as yf
from sklearn.preprocessing import MinMaxScaler
```

We start by gathering the data and defining the output as next day's close price:


```
In [2]: # Read the CSV file
file_path = 'AAPL.csv'
df = pd.read_csv(file_path, delimiter=';', decimal=',', parse_dates=
['Date'], date_format='%m/%d/%y')
df = df.sort_values('Date')
df = df.set_index('Date')

df['Output'] = df.Price.shift(-1)
# Reset the index to make Date a regular column
df = df.reset_index(names=['Date'])
# Set 'Date' as index and keep only 'Ret' column
df = df.dropna()

df.head()
```

```
Out[2]:
```

	Date	Price	Volume	Change	% Change	% Return	Total Return (Gross)	Cumulative Return %	Open	High	Low	Cumulative Change %	Output
2773	1991-12-02	199.6	118899320	0.01	1.97	1.97	0.48	276.80	0.45	0.46	0.45	260.13	0.45
2774	1991-12-03	199.5	102860910	-0.01	-2.42	-2.42	0.47	267.70	0.46	0.46	0.45	251.44	0.45
2776	1991-12-05	199.5	99198490	0.00	-0.99	-0.99	0.47	264.06	0.45	0.46	0.44	247.96	0.44
2777	1991-12-06	199.4	196985780	-0.01	-2.50	-2.50	0.46	254.96	0.44	0.44	0.43	239.26	0.44
2778	1991-12-09	199.4	97832080	0.00	0.77	0.77	0.46	257.69	0.44	0.45	0.44	241.87	0.44

```
In [3]: # Convert 'Date' column to datetime for proper filtering
df['Date'] = pd.to_datetime(df['Date'])

# Filter for dates from 2010 onwards
df = df[df['Date'] >= '2010-01-01'].reset_index(drop=True)

print("Length of data frame: ", len(df))
df.head()
```

Length of data frame: 3852

Out[3]:

	Date	Price	Volume	Change	% Change	% Return	Total Return (Gross)	Cumulative Return %	Open	High	Low	Cumulative Change %	Output
0	2010-01-04	7.64	493728720.0	0.12	1.56	1.56	8.37	6.427,19	7.62	7.66	7.58	5.857,29	7.66
1	2010-01-05	7.66	601904560.0	0.01	0.17	0.17	8.39	6.438,47	7.66	7.70	7.62	5.867,59	7.53
2	2010-01-06	7.53	552158800.0	-0.12	-1.59	-1.59	8.26	6.334,47	7.66	7.69	7.53	5.772,67	7.52
3	2010-01-07	7.52	477129720.0	-0.01	-0.18	-0.18	8.24	6.322,57	7.56	7.57	7.47	5.761,81	7.57
4	2010-01-08	7.57	447876700.0	0.05	0.66	0.66	8.29	6.365,27	7.51	7.57	7.47	5.800,78	7.50

2. Validation, train and test sample definition

Next, we proceed to define our training, test, and validation sets. Note that we will train using the 98% of the sample. The validation set will be 20% of the training test. We also need to define a 'window' of observations that we are going to take as a sequence input to our network. In this case we choose a window of 20, which means that the last 20 prices (observations) will be incorporated into the sequence of inputs for the network. Note that not all of these may end up playing a big role in the prediction (this is for the training to decide), but we will give the network the possibility to consider up to 20 past prices.

Also, note that these, and others, parameters can be included as hyperparameters of the models that we can tune to achieve maximum performance of the algorithm. This is something we leave for future implementations of the Neural Network.

```

In [4]: # First, we deal with the definition of the training, test, and validations
sets sizes
val_split = 0.2
train_split = 0.98
train_size = int(len(df) * train_split)
val_size = int(train_size * val_split)
test_size = int(len(df) - train_size)

window_size = 20 # How many daily prices do I want to incorporate in the
input sequence for the network?

ts = test_size
split_time = len(df) - ts
test_time = df.iloc[split_time + window_size :, 0:1].values

# Second, we actually define the input matrix for train (X_train) and test
(X_test), as well as the output vector (y_train and y_test). See how we
select 'Adj close' column for the Xdf matrix.
# Note that these will need further work afterwards, in order to
incorporate the info from the 'sequence' of observations
Xdf, ydf = df.iloc[:, 1:2], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")

y_train_set = y[:split_time]
y_test_set = y[split_time:]

X_train_set = X[:split_time]
X_test_set = X[split_time:]

n_features = X_train_set.shape[1]

# Third, we proceed with scaling inputs to the model. Note how this is
specially important now (compare to past tasks) because we are no longer
dealing with returns, but with prices!
scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train_set)
X_train_set_scaled = scaler_input.transform(X_train_set)
X_test_set_scaled = scaler_input.transform(X_test_set)

mean_ret = np.mean(y_train_set)

scaler_output = MinMaxScaler(feature_range=(-1, 1))
y_train_set = y_train_set.values.reshape(len(y_train_set), 1)
y_test_set = y_test_set.values.reshape(len(y_test_set), 1)
scaler_output.fit(y_train_set)
y_train_set_scaled = scaler_output.transform(y_train_set)

# Lastly, because we want a time series with up to 20 (window_size) past
observations, we need to append these observations into our matrix/vectors!
training_time = df.iloc[:split_time, 0:1].values

X_train = []
y_train = []

for i in range(window_size, y_train_set_scaled.shape[0]):
    X_train.append(X_train_set_scaled[i - window_size : i, :])
    y_train.append(y_train_set_scaled[i])

```

```

X_train, y_train = np.array(X_train), np.array(y_train)

print("Size of X vector in training:", X_train.shape)
print("Size of Y vector in training:", y_train.shape)

X_test = []
y_test = y_test_set

for i in range(window_size, y_test_set.shape[0]):
    X_test.append(X_test_set_scaled[i - window_size : i, :])

X_test, y_test = np.array(X_test), np.array(y_test)
print("Size of X vector in test:", X_test.shape)
print("Size of Y vector in test:", y_test.shape)
print("Number of features in the model: ", n_features)

```

```

Size of X vector in training: (3754, 20, 1)
Size of Y vector in training: (3754, 1)
Size of X vector in test: (58, 20, 1)
Size of Y vector in test: (78, 1)
Number of features in the model: 1

```

3. Defining the RNN model

Once we have pre-processed the input and output data of our network, let's build it!

We will start with a very simple RNN that considers 5 hidden layers (4 simple RNN layers + 1 final dense layer). Each of the RNN layers will have 50 neurons. We apply a Dropout of 20% after each of the RNN layers. Finally, before the single-unit final output layer, we will apply a dense layer with 10 neurons. As you know already, this architecture is kind of random. We will play around with the composition of the network once we move to more advanced models through hyperparameter tuning.

A novel thing of RNNs is that we can set the 'return_sequences' argument to True or False, depending on whether we want to pass the last output (if False) or the whole sequence (if True) to the next layer. More info on this here:

https://www.tensorflow.org/api_docs/python/tf/keras/layers/SimpleRNN

```
In [5]: import tensorflow as tf
```

```
model = tf.keras.models.Sequential([
    tf.keras.layers.SimpleRNN(
        50, return_sequences=True, input_shape=(X_train.shape[1],
n_features)
    ),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=True),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=True),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=False),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10),
    tf.keras.layers.Dense(1),
])
```

2025-05-14 13:05:07.100468: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-05-14 13:05:07.148245: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

2025-05-14 13:05:08.476403: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

/usr/local/lib/python3.11/site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

So, how does the model architecture look like?

```
In [6]: model.summary()
```

Model: "sequential"

Layer (type) Param #	Output Shape	
simple_rnn (SimpleRNN) 2,600	(None, 20, 50)	
dropout (Dropout) 0	(None, 20, 50)	
simple_rnn_1 (SimpleRNN) 5,050	(None, 20, 50)	
dropout_1 (Dropout) 0	(None, 20, 50)	
simple_rnn_2 (SimpleRNN) 5,050	(None, 20, 50)	
dropout_2 (Dropout) 0	(None, 20, 50)	
simple_rnn_3 (SimpleRNN) 5,050	(None, 50)	
dropout_3 (Dropout) 0	(None, 50)	
dense (Dense) 510	(None, 10)	
dense_1 (Dense) 11	(None, 1)	

Total params: 18,271 (71.37 KB)

Trainable params: 18,271 (71.37 KB)

Non-trainable params: 0 (0.00 B)

Note how here the number of parameters to be trained by the model is defined a little bit different at each step. For example, for the second RNN layer, note that we have a total of 5,050 parameters. Where does this number come from?

Well, $50 \times 50 + 50 \times 50 + 50 \times 1 = 5,050$ parameters. In other words, outputs from previous layer (50) times neurons in this layer (50) plus the neurons in the layer times themselves (50×50), plus the bias term vector (50×1).

Where does the 2,600 parameters of the first layer come from? Well, 50×50 (neurons in the layer) + 50×1 (neurons times number of model features) + 50×1

(bias terms) = 2,600

Alike we did for other models, if we sum up all the parameters in the different layers we end up with 18,271 parameters to be trained in this basic model.

4. Model training

Now, we can proceed with model training. As you will see, there is nothing really new about this and basically training works in exactly the same way once we have defined our hidden layers as 'recurrent' ones.

```
In [7]: from keras.callbacks import EarlyStopping

hp_lr = 1e-4
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=hp_lr),
    loss="mean_absolute_error"
)

es = EarlyStopping(
    monitor="val_loss", mode="min", verbose=1, patience=10,
    restore_best_weights=True
)

# fit the models
model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=500,
    batch_size=64,
    verbose=1,
    callbacks=[es],
)
```

Epoch 1/500			
47/47		5s	27ms/step - loss: 0.8755 - val_loss: 0.5018
Epoch 2/500			
47/47		2s	15ms/step - loss: 0.7682 - val_loss: 0.3504
Epoch 3/500			
47/47		1s	15ms/step - loss: 0.6676 - val_loss: 0.2727
Epoch 4/500			
47/47		1s	17ms/step - loss: 0.6242 - val_loss: 0.2808
Epoch 5/500			
47/47		1s	15ms/step - loss: 0.5754 - val_loss: 0.1098
Epoch 6/500			
47/47		1s	15ms/step - loss: 0.5630 - val_loss: 0.1120
Epoch 7/500			
47/47		1s	15ms/step - loss: 0.5124 - val_loss: 0.1234
Epoch 8/500			
47/47		1s	15ms/step - loss: 0.4827 - val_loss: 0.1160
Epoch 9/500			
47/47		1s	15ms/step - loss: 0.4461 - val_loss: 0.0858
Epoch 10/500			
47/47		1s	15ms/step - loss: 0.4200 - val_loss: 0.1336
Epoch 11/500			
47/47		1s	15ms/step - loss: 0.3870 - val_loss: 0.0733
Epoch 12/500			
47/47		1s	15ms/step - loss: 0.3677 - val_loss: 0.0799
Epoch 13/500			
47/47		1s	15ms/step - loss: 0.3573 - val_loss: 0.0771
Epoch 14/500			
47/47		1s	16ms/step - loss: 0.3136 - val_loss: 0.1388
Epoch 15/500			
47/47		1s	15ms/step - loss: 0.3070 - val_loss: 0.0667
Epoch 16/500			
47/47		1s	15ms/step - loss: 0.2891 - val_loss: 0.2095
Epoch 17/500			
47/47		1s	15ms/step - loss: 0.2694 - val_loss: 0.0884
Epoch 18/500			
47/47		1s	15ms/step - loss: 0.2530 - val_loss: 0.0768
Epoch 19/500			
47/47		1s	15ms/step - loss: 0.2293 - val_loss: 0.1024
Epoch 20/500			
47/47		1s	15ms/step - loss: 0.2189 - val_loss: 0.0873
Epoch 21/500			
47/47		1s	15ms/step - loss: 0.1981 - val_loss: 0.0665
Epoch 22/500			
47/47		1s	15ms/step - loss: 0.1845 - val_loss: 0.0676
Epoch 23/500			
47/47		1s	15ms/step - loss: 0.1743 - val_loss: 0.0637
Epoch 24/500			
47/47		1s	15ms/step - loss: 0.1633 - val_loss: 0.0608
Epoch 25/500			
47/47		1s	15ms/step - loss: 0.1587 - val_loss: 0.0647
Epoch 26/500			
47/47		1s	15ms/step - loss: 0.1468 - val_loss: 0.0677
Epoch 27/500			
47/47		1s	15ms/step - loss: 0.1421 - val_loss: 0.0674
Epoch 28/500			
47/47		1s	15ms/step - loss: 0.1300 - val_loss: 0.0644
Epoch 29/500			
47/47		1s	15ms/step - loss: 0.1202 - val_loss: 0.0639
Epoch 30/500			
47/47		1s	15ms/step - loss: 0.1203 - val_loss: 0.0648


```
Epoch 31/500
47/47 ————— 1s 15ms/step - loss: 0.1103 - val_loss: 0.0630
Epoch 32/500
47/47 ————— 1s 15ms/step - loss: 0.1074 - val_loss: 0.0618
Epoch 33/500
47/47 ————— 1s 15ms/step - loss: 0.1037 - val_loss: 0.0761
Epoch 34/500
47/47 ————— 1s 16ms/step - loss: 0.0978 - val_loss: 0.0626
Epoch 34: early stopping
Restoring model weights from the end of the best epoch: 24.
```

```
Out[7]: <keras.src.callbacks.history.History at 0x714a7b4dfe90>
```

Next, as usual, we will visualize our predictions in the test sample, undergoing the usual inverse transform to un-do the scaling process:

```
In [8]: prediction = model.predict(X_test)
prediction = scaler_output.inverse_transform(prediction)
prediction = prediction.flatten()

values = np.array(y_test[window_size:])
values = values.flatten()
```

```
2/2 ————— 1s 310ms/step
```

```
In [9]: df_predictions = pd.DataFrame(
        {"Date": test_time.flatten(), "Pred RNN": prediction, "values":
        values}
    )
df_predictions.Date = pd.to_datetime(df_predictions.Date, format="%Y-%m-%d")
df_predictions.head()
```

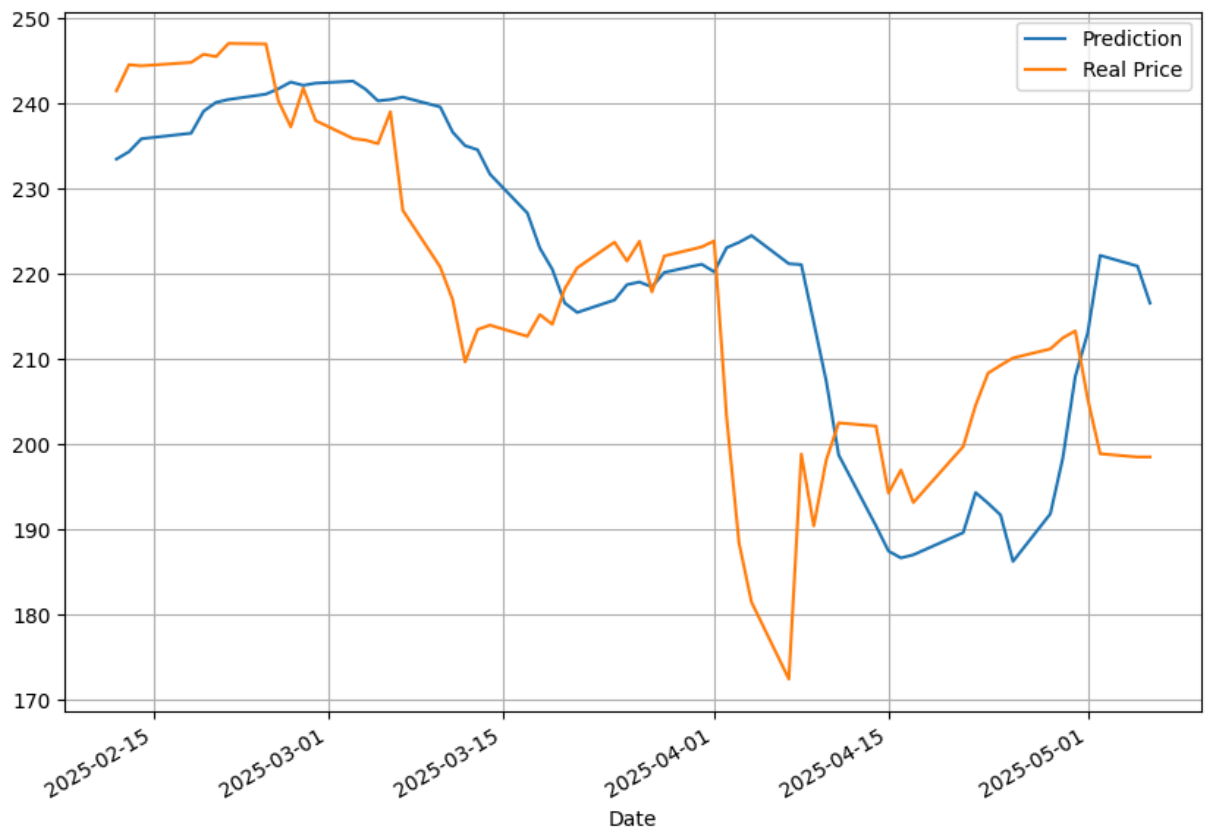
```
Out[9]:
```

	Date	Pred RNN	values
0	2025-02-12	233.512192	241.529999
1	2025-02-13	234.372055	244.600006
2	2025-02-14	235.897598	244.470001
3	2025-02-18	236.552780	244.869995
4	2025-02-19	239.142578	245.830002

And, finally, in a plot:

```
In [10]: import matplotlib.pyplot as plt

plt_1 = plt.figure(figsize=(10, 7))
ax = plt.gca()
df_predictions.plot(x="Date", y="Pred RNN", label="Prediction", ax=ax)
df_predictions.plot(x="Date", y="values", label="Real Price", ax=ax)
plt.grid()
plt.show()
```



As you can see, our model does not perform so well for predicting the time-series of prices given a sequence. It kind of 'guesses' part of the trend, but is far from being a nice tool to closely predict future stock prices. All rights reserved WQU WorldQuant University QQQQ

5. Conclusion

In this lesson we have implemented our first Recurrent Neural Network (RNN) using a sequence of 20 past prices to predict future ones. We have seen that the predictive performance of this model is not so great after all. But there are lots of things to be done in terms of improving the model. This is what we will tackle in the next lessons, which will also serve as a good excuse to introduce the details of how forward and backprop work in RNN models.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Zhang, Ashton, et al. *Dive into Deep Learning*. <https://d2l.ai/d2l-en.pdf>.

- **Required pages:** Chapter 9: Sections 9.5.3 and 9.7
- **Estimated reading time:** 25 minutes

Module 4: Lesson 3

Backpropagation in RNNs



Outline

- ▶ Backprop in RNNs: Clipping gradients
- ▶ Dealing with backprop through time
- ▶ Backprop for RNNs in detail

Backprop in RNNs: Clipping gradients

Backpropagation in recurrent neural networks (RNNs) poses a bunch of challenges. We will use this lesson to identify them and offer some simple (and not so simple) ways to mitigate them.

- ▶ The length of the sequence in RNNs introduces a new dimension for our deep neural networks.
- ▶ This means that in each iteration in backpropagation we have to calculate different gradients through time.
- ▶ The consequence is a potentially huge chain of matrix-products that may result in numerical instability.
- ▶ The latter will cause something that, unfortunately, we are already familiar with: **vanishing and exploding gradients**.

A usual 'hack' used in practice to avoid the vanishing/exploding gradients problem is **gradient clipping**:

- Is essentially a way to force gradients to take smaller values.
- In each update of the gradient: $x \leftarrow x - \eta g$, g will be 'clipped' by:

$$g \leftarrow \min \left(1, \frac{\theta}{\|g\|} \right) g$$

Dealing with backprop through time

Applying backpropagation in RNNs is called **backpropagation through time**. Suppose we are in a simple RNN where hidden and output layers at time t are defined as:

$$h_t = f(x_t, h_{t-1}, w_h) \quad \text{and} \quad o_t = g(h_t, w_o)$$

The loss function across T time steps is:

$$L(x_1, \dots, x_T, y_1, \dots, y_T, w_h, w_o) = \frac{1}{T} \sum_{t=1}^T l(y_t, o_t)$$

Backprop requires us to compute several gradients, for which we can apply the chain rule:

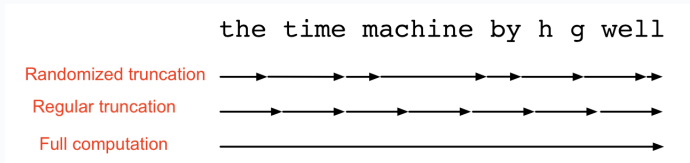
$$\begin{aligned} \frac{\partial L}{\partial w_h} &= \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial w_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial g(h_t, w_o)}{\partial h_t} \frac{\partial h_t}{\partial w_h} \\ \frac{\partial h_t}{\partial w_h} &= \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial f(x_j, h_{j-1}, w_h)}{\partial h_{j-1}} \right) \frac{\partial f(x_i, h_{i-1}, w_h)}{\partial w_h} \end{aligned}$$

While the chain rule is very useful for the last computation, it can get very long as t increases!

Dealing with backprop through time

There are a bunch of strategies that we can use to deal with the computation of gradients and chain rules when t is very large:

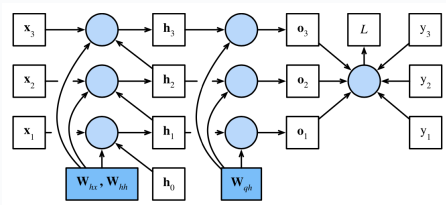
- ▶ Full computation
→ Compute 'the full sum', being aware that things can 'blow up' if t is large
- ▶ Truncating time steps
→ Truncate the sum after τ steps
- ▶ Randomized truncation
→ Use a random variable, which is correct in expectation but truncates the sequence



Backprop for RNNs in detail

The detailed calculations embedded in backprop for RNNs is described in detail in section 9.7.2 in chapter 9 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

- ▶ We direct you to that section (part of the required readings) for the full explanation.
- ▶ Here, we simply review a few of the steps part of the process:



Even in this very simply network we will have to, using the chain rule, calculate (in backprop order):

$$\frac{\partial L}{\partial o_t} \rightarrow \frac{\partial L}{\partial W_{qh}} \rightarrow \frac{\partial L}{\partial h_t} \rightarrow \frac{\partial L}{\partial W_{hx}} \rightarrow \frac{\partial L}{\partial W_{hh}} \rightarrow \dots$$

Summary of Lesson 3

In Lesson 3, we have:

- ▶ Identified the sources of vanishing/exploding gradient problems in RNNs
- ▶ Presented several options to deal with long sequences in RNNs
- ▶ Gone through backprop through time in detail

⇒ **References for this lesson:**

Chapter 9, Sections 9.5.3 and 9.7 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will improve different aspects of the RNN network and revisit the example from Lesson 2, extending it to the multivariate case.

Module 4: Lesson 4

Multivariate RNNs

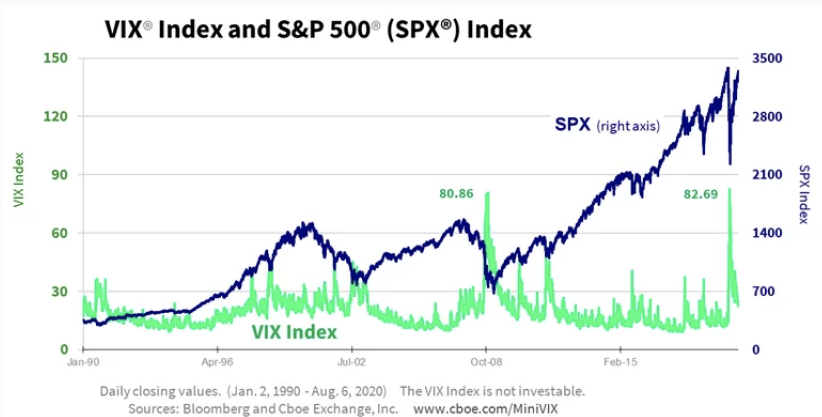


Outline

- ▶ The S&P500 and the VIX
- ▶ Multivariate RNNs

The S&P500 and the VIX

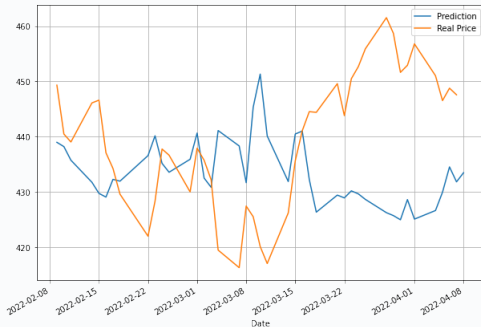
For our next prediction problem, we will use a multivariate RNN. What does this mean? Simply that it contains more than one input feature. This makes a lot of sense, especially if we can get our hands on 'leading' data:



The S&P500 and the VIX

In our multivariate RNN, we will aim to predict the next day's price of the SPY (ETF over the S&P500 index) using past prices of SPY and prices (or levels) of the VIX.

Sadly, our results are not so good...



Note that the training of this network requires some random initialization so the graph may look different!

Summary of Lesson 4

In Lesson 4, we:

- ▶ Explored the relationship between a market index such as S&P500 and its volatility index (VIX)
- ▶ Used both inputs in an RNN
- ▶ Worked with the multivariate case of RNNs

TO DO NEXT: Go to the accompanying Python notebook to implement your first multivariate RNN!

This is all for Module 4. In the next module, we will deal with two 'modern' types of RNNs: gated recurrent units (GRUs) and long short-term memory (LSTM). See you there!

DEEP LEARNING FOR FINANCE

MODULE 4 | LESSON 4

Multivariate RNNs

Reading Time	45 minutes
Prior Knowledge	RNN, Deep Neural Network, Market timing
Keywords	Multivariate RNN, VIX, S&P500

1. Introduction: S&P500 index and the VIX

It is now time to augment our deep RNNs to consider more than one series of inputs. In this lesson we revisit the market timing example of Lesson 2, but we will now predict stock prices using past information (prices) about the security as well as a leading indicator variable. Specifically, we will try to predict the prices of an Exchange Traded Fund (ETF) that replicates the composition and behavior of the S&P500 index in the U.S., the famous 'SPY'. And we will use for that not only past prices of SPY, but also the prices (levels) of a leading indicator of S&P500 index: the VIX index.

The VIX is a volatility index of the S&P500 market index. It is constructed using the implied volatility from a portfolio of 30-day constant-maturity options on S&P500 index. Since we all well know at this point, the fact that VIX index is constructed using options-implied volatility information means that it reflects investor's expectations on the underlying asset, the S&P500 index (Ticker symbol 'SPX'). More information on the VIX, and how it can be useful in the next few bullet points (please note these materials are only informative, not investment recommendation!):

- More information about the VIX index by the 'creators', Chicago Board Options Exchange (CBOE): https://www.cboe.com/tradable_products/vix/faqs/
- How investors & traders use the VIX: <https://optionshawk.com/how-to-use-the-vix-as-a-market-signal/>
- A web were you can consult the 'term structure' of Future contracts over VIX with different maturities: <http://vixcentral.com/>

- How to interpret the VIX term structure: <https://www.projectfinance.com/vix-term-structure/>
- An interesting paper on how to use VIX term structure to predict recessions: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3943982
- A great academic paper on how VIX term structure predicts risk premia: https://www.cambridge.org/core/services/aop-cambridge-core/content/view/56572D1F060448571BD8F597C732D9C3/S0022109017000825a.pdf/risk_premia_and_the_vix_term_structure.pdf
- On using VIX in investing under a formal, systematic approach: <https://joi.pm-research.com/content/23/3/130.short>

When you come to think about it, if VIX really works as a powerful leading indicator of SPX, adding it as an input to our RNN prediction problem should be like 'cheating'. If we think about another type of prediction task suited for RNNs, such as language models, it would be like asking the model to predict how a sentence would end knowing what are the next few words. It should simplify the prediction task a lot!

Spoiler alert! It will not. Or, at least, it will not necessarily do it. For example, what if investors already looked at VIX prices to decide on their trading in the S&P500? In that case, it is likely that adding the VIX to our input matrix set could be even redundant, as this information is already incorporated into prices themselves.

Of course, it may be the case that investors (or at least some of them) are oblivious to information contained in VIX. For starters, it is likely that investors trading the S&P500 index and its components are inherently different from those trading options on the SPX. These differences can arise not only in terms of how informed are each of these guys, but also their investment horizon, risk-aversion, ... All in all, these features (and others, like limits to arbitrage) cast doubts (at least ex-ante) on whether S&P500 incorporates data on VIX and the potential predictive power of VIX as a leading indicator.

So... let's resolve this question!

2. Data gathering

As usual, we will start with the data gathering process. Note how in this case we do not only gather daily stock prices from our S&P500 index ETF, SPY, from Yahoo! finance, but also on the daily prices (levels) of VIX. We will be working, as usual, with daily prices and trying to predict what is the next price that SPY will take.

```
In [1]: import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Read the CSV file
file_path = 'SPY.csv'
spy_df = pd.read_csv('SPY.csv', sep=';', decimal=',')
spy_df['Date'] = pd.to_datetime(spy_df['Date'], format='%m/%d/%y')
# Create a column for next day's SPY price (Output)
spy_df['Output'] = spy_df['Price'].shift(-1)

spy_df = spy_df.dropna()

spy_df.head()
```

```
Out[1]:
```

	Date	Price	CVol	% Change	Open	Low	High	NAV	Total Return (Gross)	% Return	Cumulative Return %	Output
1	2025-05-13	586.84	67947234	0.66	583.41	582.84	589.08	587.02	776.06	0.66	596.39	582.99
2	2025-05-12	582.99	78993570	3.30	581.47	577.04	583.00	582.79	770.97	3.30	591.82	564.34
3	2025-05-09	564.34	37603434	-0.13	566.48	562.76	567.50	564.37	746.30	-0.13	569.69	565.06
4	2025-05-08	565.06	65130793	0.70	565.24	561.70	570.31	564.68	747.26	0.70	570.55	561.15
5	2025-05-07	561.15	55733230	0.42	560.15	556.04	563.82	561.42	742.09	0.42	565.91	558.80

```
In [2]: vix_df = pd.read_csv('VIX.csv', sep=';', decimal=',')
vix_df['Date'] = pd.to_datetime(vix_df['Date'], format='%m/%d/%y')
vix_df.head()
```

Out[2]:

	Date	Price	CVol	Change	% Change	Total Return (Gross, Unhedged)	Cumulative Return % (Gross, Unhedged)	Open	High	Low
0	2025-05-14	18.22	NaN	NaN	NaN	NaN	NaN	NaN	18.22	18.22
1	2025-05-13	18.22	NaN	-0.17	-0.92	NaN	NaN	18.69	18.85	17.65
2	2025-05-12	18.39	NaN	-3.51	-16.03	NaN	NaN	19.84	20.44	18.14
3	2025-05-09	21.90	NaN	-0.58	-2.58	NaN	NaN	22.40	22.82	21.83
4	2025-05-08	22.48	NaN	-1.07	-4.54	NaN	NaN	22.84	23.61	21.88

```
In [3]: # Merge the two dataframes on Date
df = pd.merge(
    spy_df[['Date', 'Price', 'Output']],
    vix_df[['Date', 'Price']],
    on='Date',
    how='inner'
)

# Rename columns for clarity
df = df.rename(columns={
    'Price_x': 'SPY_Price',
    'Price_y': 'VIX_Price'
})

# Drop any rows with NaN values (will be the last row due to shift operation)
df = df.dropna()

# Sort by date (newest first, matching the original data order)
df = df.sort_values('Date', ascending=False)

# Reset index
df = df.reset_index(drop=True)

# Reorder columns to make Output the last column
df = df[['Date', 'VIX_Price', 'SPY_Price', 'Output']]

# Display the first few rows of the final dataframe
df.head()
```

Out[3]:

	Date	VIX_Price	SPY_Price	Output
0	2025-05-13	18.22	586.84	582.99
1	2025-05-12	18.39	582.99	564.34
2	2025-05-09	21.90	564.34	565.06
3	2025-05-08	22.48	565.06	561.15
4	2025-05-07	23.55	561.15	558.80

3. Validation, train and test sample definition

Next, we proceed to define our training, test, and validation sets. Note that we will train using the 98% of the sample. The validation set will be 20% of the training test. We also need to define a 'window' of observations that we are going to take as a sequence input to our network. In this case we choose a window of 20, which means that the last 20 prices (observations) will be incorporated into the sequence of inputs for the network. In this case this means that we will have a sequence of 20 SPY prices, as well as 20 VIX prices that will be part of the training at each t .

```

In [5]: # First, we deal with the definition of the training, test, and validations
        sets sizes
        val_split = 0.2
        train_split = 0.98
        train_size = int(len(df) * train_split)
        val_size = int(train_size * val_split)
        test_size = int(len(df) - train_size)

        window_size = 20 # How many daily prices do I want to incorporate in the
                           input sequence for the network?

        ts = test_size
        split_time = len(df) - ts
        test_time = df.iloc[split_time + window_size :, 0:1].values

        # Second, we actually define the input matrix for train (X_train) and test
        (X_test), as well as the output vector (y_train and y_test).
        Xdf, ydf = (
            df.iloc[:, 1:3],
            df.iloc[:, -1],
        ) # Note how we choose close prices from both SPY and VIX in the X matrix
        X = Xdf.astype("float32")
        y = ydf.astype("float32")

        y_train_set = y[:split_time]
        y_test_set = y[split_time:]

        X_train_set = X[:split_time]
        X_test_set = X[split_time:]

        n_features = X_train_set.shape[1]

        # Third, we proceed with scaling inputs to the model.
        scaler_input = MinMaxScaler(feature_range=(-1, 1))
        scaler_input.fit(X_train_set)
        X_train_set_scaled = scaler_input.transform(X_train_set)
        X_test_set_scaled = scaler_input.transform(X_test_set)

        mean_ret = np.mean(y_train_set)

        scaler_output = MinMaxScaler(feature_range=(-1, 1))
        y_train_set = y_train_set.values.reshape(len(y_train_set), 1)
        y_test_set = y_test_set.values.reshape(len(y_test_set), 1)
        scaler_output.fit(y_train_set)
        y_train_set_scaled = scaler_output.transform(y_train_set)

        # # Lastly, because we want a time series with up to 20 (window_size) past
        observations, we need to append these observations into our matrix/vectors!
        training_time = df.iloc[:split_time, 0:1].values

        X_train = []
        y_train = []

        for i in range(window_size, y_train_set_scaled.shape[0]):
            X_train.append(X_train_set_scaled[i - window_size : i, :])
            y_train.append(y_train_set_scaled[i])

        X_train, y_train = np.array(X_train), np.array(y_train)

```

```

print(X_train.shape)
print(y_train.shape)

X_test = []
y_test = y_test_set

for i in range(window_size, y_test_set.shape[0]):
    X_test.append(X_test_set_scaled[i - window_size : i, :])

X_test, y_test = np.array(X_test), np.array(y_test)
print(X_test.shape)
print(y_test.shape)
print(n_features)

```

```
(3757, 20, 2)
```

```
(3757, 1)
```

```
(58, 20, 2)
```

```
(78, 1)
```

```
2
```

4. Defining the RNN model

Once we have pre-processed the input and output data of our network, let's build it!

Our network architecture will be identical to the one from Lesson 2. We consider a simple RNN 5 hidden layers (4 simple RNN layers + 1 final dense layer). Each of the RNN layers will have 50 neurons. We apply a Dropout of 20% after each of the RNN layers. Finally, before the single-unit final output layer, we will apply a dense layer with 10 neurons.

As we already mentioned, this choice of architecture is as good as random. We leave to you to play around with the architecture and check how predictions vary.

Remember that a good resource to have nearby when implementing any Deep Neural Network is the TensorFlow/Keras documentation:

<https://www.tensorflow.org/guide/keras/rnn?hl=en>

```
In [6]: import tensorflow as tf
```

```
model = tf.keras.models.Sequential([
    tf.keras.layers.SimpleRNN(
        50, return_sequences=True, input_shape=(X_train.shape[1],
n_features)
    ),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=True),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=True),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=False),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10),
    tf.keras.layers.Dense(1),
])
```

2025-05-14 11:13:41.036553: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-05-14 11:13:41.077864: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

2025-05-14 11:13:42.152523: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

/usr/local/lib/python3.11/site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

Let's now check how our model looks like:

```
In [7]: model.summary()
```

Model: "sequential"

Layer (type) Param #	Output Shape	
simple_rnn (SimpleRNN) 2,650	(None, 20, 50)	
dropout (Dropout) 0	(None, 20, 50)	
simple_rnn_1 (SimpleRNN) 5,050	(None, 20, 50)	
dropout_1 (Dropout) 0	(None, 20, 50)	
simple_rnn_2 (SimpleRNN) 5,050	(None, 20, 50)	
dropout_2 (Dropout) 0	(None, 20, 50)	
simple_rnn_3 (SimpleRNN) 5,050	(None, 50)	
dropout_3 (Dropout) 0	(None, 50)	
dense (Dense) 510	(None, 10)	
dense_1 (Dense) 11	(None, 1)	

Total params: 18,321 (71.57 KB)

Trainable params: 18,321 (71.57 KB)

Non-trainable params: 0 (0.00 B)

Now, the number of parameters in the first layer of the model changes a little bit. Where does the 2,650 come from?

Well, 50×50 (neurons in the layer) + 50×2 (neurons times number of model features) + 50×1 (bias terms) = 2,650

Note how the total model parameters is now 18,321. Only 50 more (because there is 1 more input feature) than the similar model from Lesson 2. Finally, note how the only impact on model parameters from incorporating 1 more feature comes in the first layer of the model. After that, there is no increase in the

trainable parameters resulting from considering more input features. This is also important to take into account when considering how the 'size' of the model may impact the performance of backprop and gradient descent algorithm: It is much more about the 'length' of the input sequence than the number of input features themselves!

5. Model training

Now, we can proceed with model training.

```
In [8]: from keras.callbacks import EarlyStopping

hp_lr = 1e-4
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=hp_lr),
    loss="mean_absolute_error"
)

es = EarlyStopping(
    monitor="val_loss", mode="min", verbose=1, patience=10,
    restore_best_weights=True
)

# fit the models
model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=500,
    batch_size=64,
    verbose=1,
    callbacks=[es],
)
```

```

Epoch 1/500
47/47 ————— 4s 21ms/step - loss: 0.6665 - val_loss: 0.3481
Epoch 2/500
47/47 ————— 1s 13ms/step - loss: 0.5605 - val_loss: 0.1852
Epoch 3/500
47/47 ————— 1s 12ms/step - loss: 0.4981 - val_loss: 0.1422
Epoch 4/500
47/47 ————— 1s 13ms/step - loss: 0.4719 - val_loss: 0.1242
Epoch 5/500
47/47 ————— 1s 13ms/step - loss: 0.4521 - val_loss: 0.2171
Epoch 6/500
47/47 ————— 1s 13ms/step - loss: 0.4232 - val_loss: 0.1209
Epoch 7/500
47/47 ————— 1s 12ms/step - loss: 0.3865 - val_loss: 0.0979
Epoch 8/500
47/47 ————— 1s 13ms/step - loss: 0.3772 - val_loss: 0.1061
Epoch 9/500
47/47 ————— 1s 12ms/step - loss: 0.3500 - val_loss: 0.0619
Epoch 10/500
47/47 ————— 1s 14ms/step - loss: 0.3317 - val_loss: 0.1167
Epoch 11/500
47/47 ————— 1s 14ms/step - loss: 0.3162 - val_loss: 0.0567
Epoch 12/500
47/47 ————— 1s 12ms/step - loss: 0.3087 - val_loss: 0.1183
Epoch 13/500
47/47 ————— 1s 12ms/step - loss: 0.2763 - val_loss: 0.1875
Epoch 14/500
47/47 ————— 1s 12ms/step - loss: 0.2662 - val_loss: 0.0869
Epoch 15/500
47/47 ————— 1s 12ms/step - loss: 0.2599 - val_loss: 0.1384
Epoch 16/500
47/47 ————— 1s 13ms/step - loss: 0.2407 - val_loss: 0.1587
Epoch 17/500
47/47 ————— 1s 14ms/step - loss: 0.2301 - val_loss: 0.0672
Epoch 18/500
47/47 ————— 1s 12ms/step - loss: 0.2247 - val_loss: 0.0813
Epoch 19/500
47/47 ————— 1s 12ms/step - loss: 0.2118 - val_loss: 0.1060
Epoch 20/500
47/47 ————— 1s 12ms/step - loss: 0.2041 - val_loss: 0.0615
Epoch 21/500
47/47 ————— 1s 13ms/step - loss: 0.1937 - val_loss: 0.1089
Epoch 21: early stopping
Restoring model weights from the end of the best epoch: 11.

```

Out[8]: <keras.src.callbacks.history.History at 0x703649975850>

Then we can make our predictions on the test sample based on the trained model parameters:

```

In [9]: prediction = model.predict(X_test)
prediction = scaler_output.inverse_transform(prediction)
prediction = prediction.flatten()

values = np.array(y_test[window_size:])
values = values.flatten()

```

2/2 ————— 1s 259ms/step

```
In [10]: df_predictions = pd.DataFrame(
    {"Date": test_time.flatten(), "Pred RNN": prediction, "values":
    values}
    )
df_predictions.Date = pd.to_datetime(df_predictions.Date, format="%Y-%m-
%d")
df_predictions.head()
```

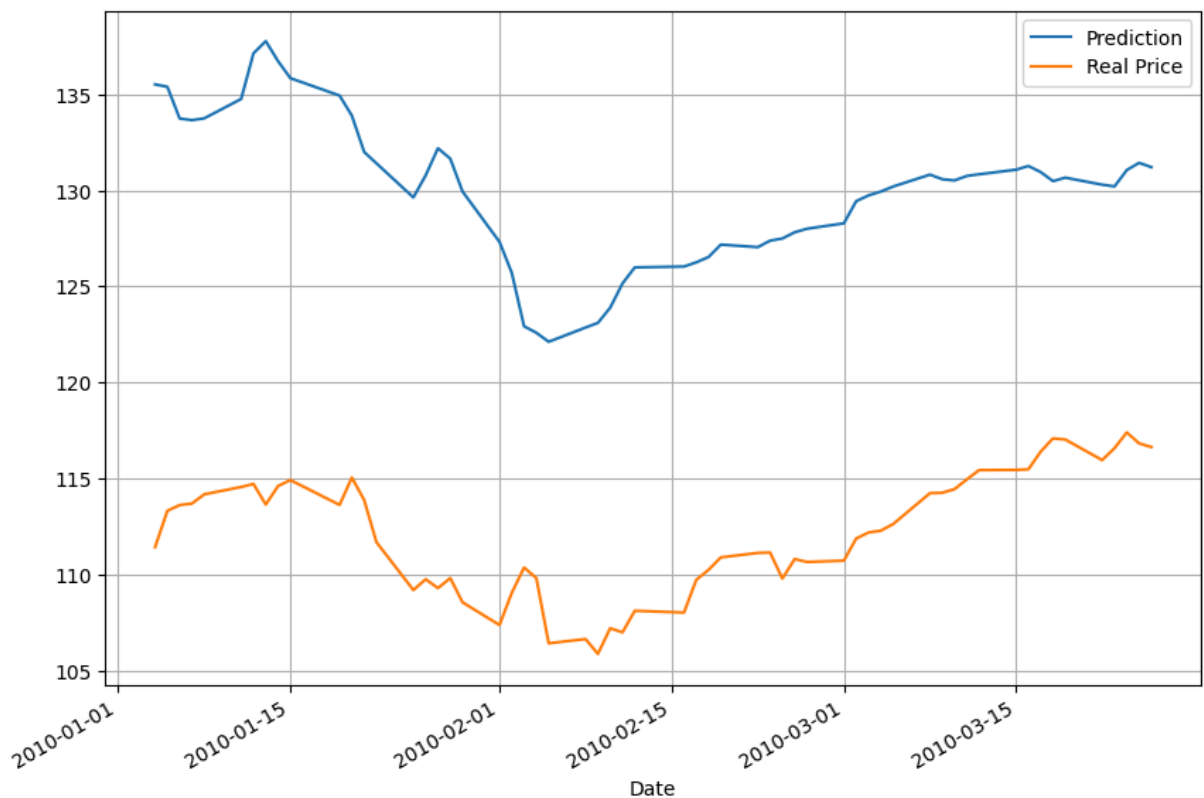
```
Out[10]:
```

	Date	Pred RNN	values
0	2010-03-26	131.208176	116.650002
1	2010-03-25	131.434296	116.839996
2	2010-03-24	131.056564	117.410004
3	2010-03-23	130.207977	116.589996
4	2010-03-22	130.297607	115.970001

And visualize it,All rights reserved WQU WorldQuant University QQQQ

```
In [11]: import matplotlib.pyplot as plt

plt_1 = plt.figure(figsize=(10, 7))
ax = plt.gca()
df_predictions.plot(x="Date", y="Pred RNN", label="Prediction", ax=ax)
df_predictions.plot(x="Date", y="values", label="Real Price", ax=ax)
plt.grid()
plt.show()
```



As you can see (and please remember that we have not fixed the random seed so you may obtain different outcomes every time you train the model), the VIX

does not really improve the predictive performance of the model by a lot (sometimes it's even making it worse!).

Can you think of ways to augment and improve this model? Well, one option may be using other types of 'leading variables'. For example, some variables used in practice for this are: prices of oil futures, manufacturing PMI in the U.S., agricultural production, ...

But we would usually not like a model that uses too many inputs because it will probably lead not only to increase training time and computational resources, but also to a lower degree of understanding regarding the interplay between the different features. Bottom line, it is important to understand the basics of how models work before we start adding inputs!

6. A final note on NLP

One of the main uses of RNNs is Natural Language Processing (NLP), which is a fascinating area. Sadly, there are, yet, not so many applications of NLP to financial market problems, which is why we have not focused on language models in this module.

In the book by Zhang. et. al. (2022) that we have been using as a reference for the whole course you have much more information on NLP models in case you are interested. Specifically, Chapters 15 and 16 deal with the pre-training and direct applications of NLP models in practice. Note that RNNs are just one part of NLP models, but probably the ones that make more sense when focusing on areas such as speech recognition.

Here you can have more info for the use of NLP in finance:

- A few examples of finance-related applications of NLP:
<https://softengi.com/blog/nlp-use-cases-in-finance/>
- One important application of NLP in finance is sentiment analysis. Here you have a quick review of it: <https://research.aimultiple.com/sentiment-analysis-stock-market/>
- A paper by Mittal and Goel (Stanford) that uses Twitter sentiment analysis for stock price prediction: <http://cs229.stanford.edu/proj2011/GoelMittal-StockMarketPredictionUsingTwitterSentimentAnalysis.pdf>

These are just some examples to 'whet your appetite'. Surely you can find many more!

7. Conclusion

In this lesson, we have dealt with multivariate RNNs. This means that we have used several input features into a recursive neural network model. So far, we

have not obtained great results, though. But we have used a very tiny portion of all the available tools in deep learning and recursive networks.

In the next module, we will explore more specific types of RNN models: Gated Recurrent Units (GRUs) and Long Short-Term Memory (LSTM). See you in the next module!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.